Process Scheduling, Process Operations & Inter-Process Communication

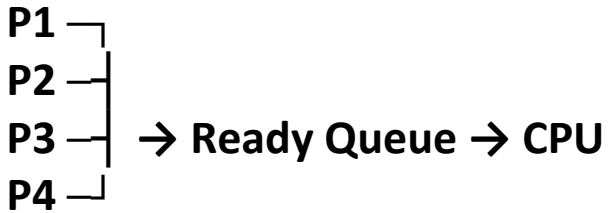
Today's Syllabus

- Process Scheduling
- Scheduling Queues
- Schedulers
- Operations on Processes
- Process Creation and Termination
- Cooperating Processes
- Inter-Process Communication (IPC)
- Shared Memory
- Message Passing

“Suppose Chrome, VS Code, Spotify and File Explorer are all running. There is only limited CPU time. Who decides which process gets the CPU?”
Answer: Operating System / CPU Scheduler

What is Process Scheduling?

Definition
Process scheduling is the activity of selecting a process from the ready queue and allocating the CPU to it.
Simple idea



The **CPU scheduler** selects one of the ready processes.

Why Do We Need Scheduling?

Suppose four processes are ready:

P1 P2 P3 P4

↓ ↓ ↓ ↓

CPU

A single CPU core cannot execute all of them simultaneously.
Therefore, the OS must decide:
Which process should run next?
This decision is made according to a **scheduling algorithm**.
Examples

- FCFS
- SJF
- SRTF
- Priority
- Round Robin

CPU-I/O Burst Cycle

A process typically alternates between CPU execution and I/O waiting.

CPU Burst
↓
I/O Burst
↓
CPU Burst
↓
I/O Burst
↓
CPU Burst
↓
Termination

Example

A media application might:

1. Use CPU
2. Read data from storage
3. Use CPU to process data
4. Wait for more data
5. Continue processing

CPU Burst vs I/O Burst

CPU Burst
Period during which the process executes instructions on the CPU.
I/O Burst
Period during which the process waits for or performs I/O activity.

| CPU | I/O | CPU | I/O | CPU |

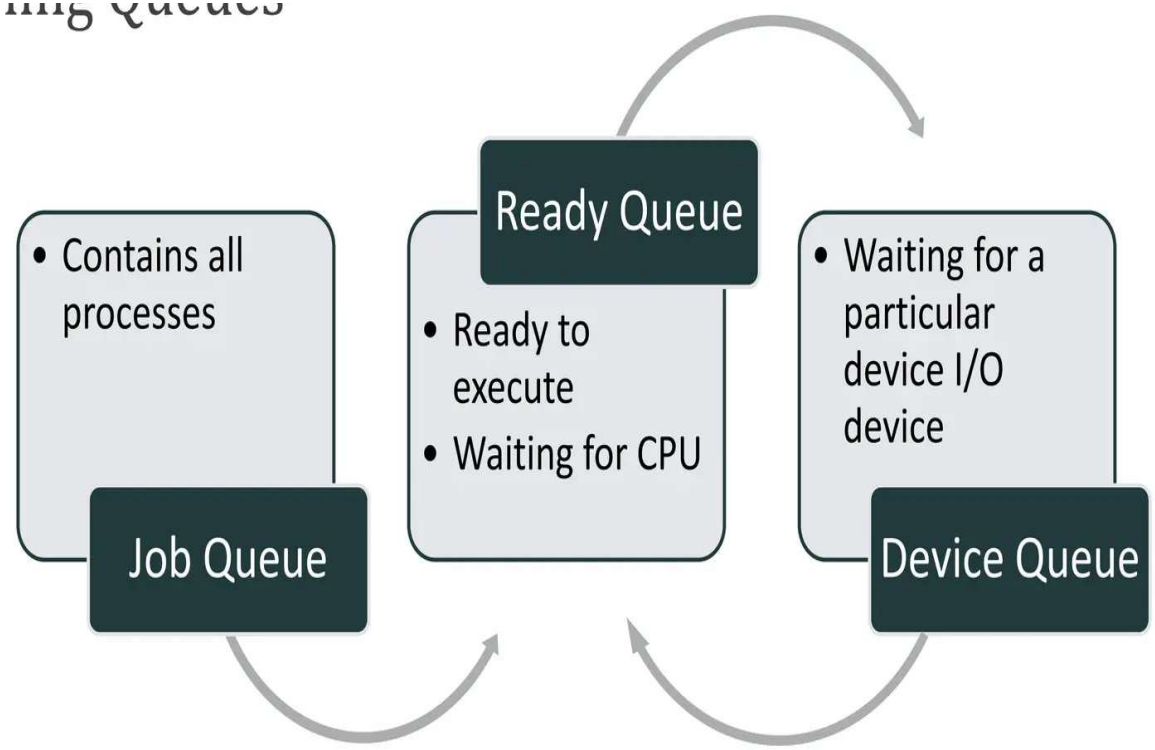
Important
CPU scheduling mainly determines:
Which ready process gets the CPU during CPU scheduling decisions.

Scheduling Queues

Processes move through different queues.

Main Queues

- Job Queue
- Ready Queue
- Device Queues



New Processes
↓
Job Queue
↓
Ready Queue
↓
CPU
↓
I/O Request
↓
Device Queue
↓
I/O Complete
↓
Ready Queue

Ready Queue

Ready Queue
Contains processes that: **are ready to execute and are waiting for CPU allocation.**
Example:

READY QUEUE



↓
CPU Scheduler
↓
CPU

Important
A process in the **Ready Queue is not currently executing**.

Device Queue

If a running process requests I/O, it may move to a device queue.
Example:
Running
↓
Disk I/O Request
↓
Disk Queue
↓
I/O Complete
↓
Ready Queue

Example
A process requests data from an SSD.
While waiting for the operation, it doesn’t need to occupy the CPU.

Schedulers

The OS may use different types of schedulers.

Three commonly discussed schedulers:

1. Long-Term Scheduler
2. Short-Term Scheduler
3. Medium-Term Scheduler

Long-Term Scheduler

Long-Term Scheduler / Job Scheduler
Selects processes from the job pool and admits them into the system for execution.
Main purpose
Controls the **degree of multiprogramming**.
Simplified

Job Pool
↓
Long-Term Scheduler
↓
Ready Queue

Characteristics
- Runs less frequently
- Can be relatively slower
- Controls how many processes enter the system

Short-Term Scheduler

Short-Term Scheduler / CPU Scheduler
Selects a process from the ready queue and allocates the CPU.

Ready Queue
↓
Short-Term Scheduler
↓
CPU

This scheduler runs **very frequently**.
Therefore, it needs to be fast.

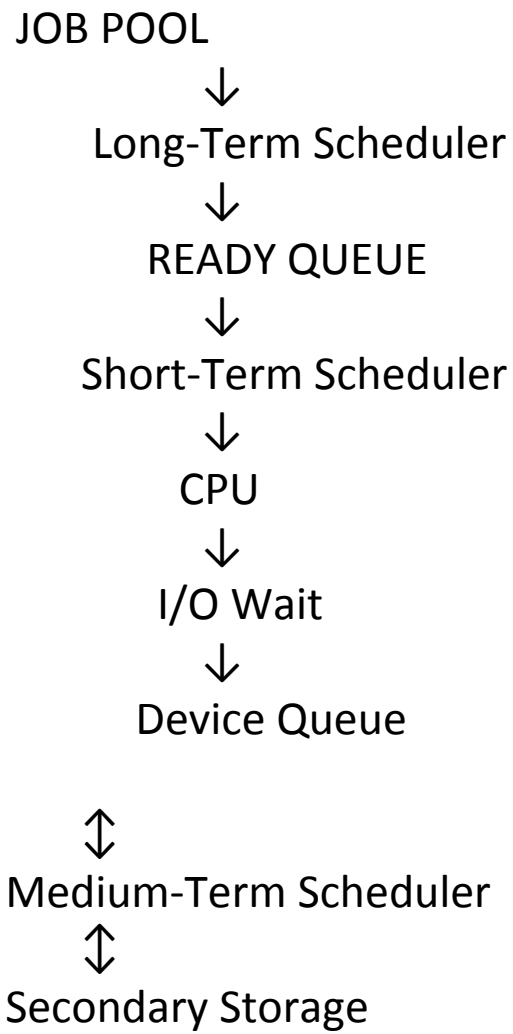
Medium-Term Scheduler

Medium-Term Scheduler
Can temporarily remove processes from memory and later bring them back.
This is associated with:
Swapping

Simplified -
Memory
↓
Swap Out
↓
Secondary Storage
↓
Swap In
↓
Memory

Purpose
Helps manage memory and the degree of multiprogramming.

Three Schedulers Together



Process Operations

The OS performs several operations on processes.

Major Operations

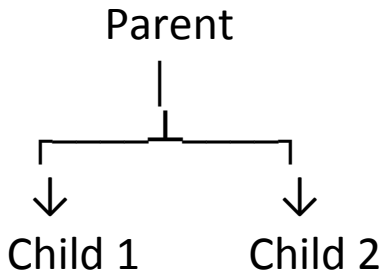
- Process creation
- Process execution
- Process suspension
- Process resumption
- Process termination

Today focus mainly on:

Creation + Termination

Process Creation

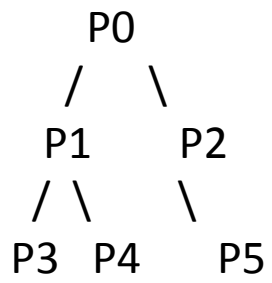
A process can create another process.
The creating process is called:
Parent Process
The newly created process is called:
Child Process



Example
A shell creates a process when you execute a command.

Process Tree

Processes can create additional processes.



This is called a:
Process Tree
Important
Every process typically has a unique **Process ID (PID)**.

Process Creation in UNIX/Linux

In Unix/Linux, process creation commonly involves:

fork()

Creates a new process.

exec()

Loads a new program into a process.

Concept

Parent
↓
fork()
↓
Child
↓
exec()
↓
New Program

Example: Shell

Suppose you type:
ls

Conceptually:

Shell
↓
Creates/starts process
↓
ls executes
↓
Output
↓
Process terminates

Process Termination

A process terminates when:

- It completes execution
- It explicitly calls an exit operation
- An error occurs
- Another process/OS terminates it

Running
↓
Execution completed
↓
Terminated

What Happens After Termination?

When a process terminates, the OS can reclaim resources such as:

- Memory
- Open resources
- Scheduling information
- Other process-associated resources

The process's entry is eventually removed from the active process-management structures.

Cooperating Processes

What is a Cooperating Process?

A process is **cooperating** if it can affect or be affected by other processes executing in the system.

Why cooperate?

- Information sharing
- Computation speedup
- Modularity
- Convenience

Example

A web browser may have multiple processes that perform different tasks.

Independent vs Cooperating Processes

Independent Process	Cooperating Process
Does not affect or depend on other processes	Can interact with other processes
Works independently	Shares data/information or coordinates
Limited interaction	Uses IPC mechanisms
Example: isolated calculation	Example: client-server application

Key Idea
Cooperating processes need a mechanism to communicate.
That mechanism is:
IPC

Inter-Process Communication

IPC

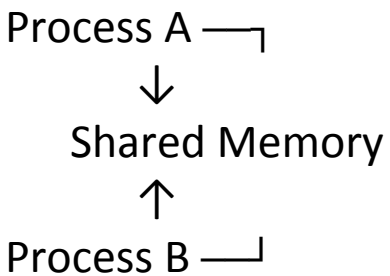
Inter-Process Communication (IPC) provides mechanisms that allow processes to exchange data and coordinate their activities.

Two fundamental models

1. Shared Memory
2. Message Passing

IPC Model 1: Shared Memory

Processes communicate by sharing a region of memory.



How it works

1. OS establishes/shared region.
2. Processes access the shared region.
3. Processes read/write data.

Advantage

Very fast communication once shared memory is established.

Challenge

Processes must coordinate access to shared data.

Shared Memory Example

Suppose two processes are working with a shared buffer:

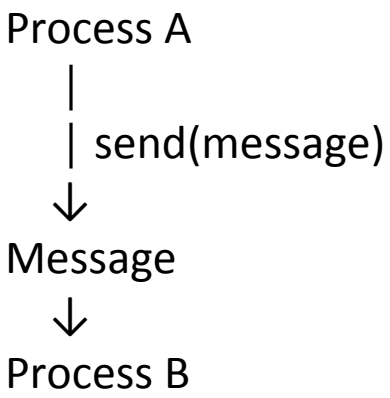
Producer
↓
Shared Buffer
↓
Consumer

Producer
Produces data.
Consumer
Consumes data.

Example:
Producer → "Hello"
Producer → "World"
↓
Shared Buffer
↓
Consumer

IPC Model 2: Message Passing

Processes communicate by sending and receiving messages.



Typical operations
send()
receive()

Example
Process A → "Data ready"
Process B → "Received"

Shared Memory vs Message Passing

Feature	Shared Memory	Message Passing
Communication	Shared memory region	Messages
Speed	Generally fast after setup	May have more communication overhead
Coordination	Explicit synchronization often needed	Communication mechanism provides structure
Suitable for	Processes on same system	Same or distributed systems
Complexity	Shared-data coordination	Message management

Important
none universally “better.”
Their suitability depends on the system and application.

IPC in Real Life

Example: Web Browser
Imagine:
Browser UI Process
↕
Network Process
↕
Renderer Process

These processes may need to exchange information.
Question
“Can they simply access each other’s private memory?”
Normally, **no**. They need controlled mechanisms such as IPC.
This is one reason process isolation and IPC are both important.

Problem-Solving Activity

Question

Suppose:

P1 = Running

P2 = Ready

P3 = Waiting for I/O

The following events occur:

Event 1

P1 requests disk I/O.

What happens?

P1: Running → Waiting

Event 2

CPU scheduler selects P2.

P2: Ready → Running

Event 3

P3's I/O completes.

P3: Waiting → Ready

Final State

P1 → Waiting

P2 → Running

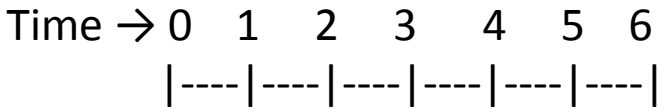
P3 → Ready

Numerical Foundation

Process	Arrival Time	Burst Time
P1	0	6
P2	1	4
P3	2	2

“At time 0, which process is available?”
P1
“At time 1, which processes are available?”
P1 and P2
“At time 2?”
P1, P2 and P3

Scheduling Timeline Activity



Mark following points:

- When P1 arrives
- When P2 arrives
- When P3 arrives

Key Terms

Process Scheduling
Selecting a ready process for CPU execution.
Scheduler
Component that makes scheduling decisions.
Ready Queue
Processes waiting for CPU allocation.
Parent Process
Process that creates another process.
Child Process
Process created by another process.
Cooperating Processes
Processes that can interact with each other.
IPC
Mechanism for process communication.

Q1. Which scheduler selects a process from the ready queue for CPU execution?
A. Long-term scheduler B. Short-term scheduler C. Medium-term scheduler D. I/O scheduler

Q2. Which scheduler controls the degree of multiprogramming by admitting jobs into the system?
A. Short-term B. Long-term C. CPU scheduler D. Device scheduler

Q3. Which mechanism allows processes to communicate through a common memory region?
A. Message passing B. Shared memory C. Context switching D. Scheduling

Q4. In a parent-child relationship, which operation is commonly associated with creating a new process in Unix/Linux?
A. read() B. fork() C. close() D. write()

Q5. A process waiting for completion of an I/O operation is generally in which state?
A. Ready B. Running C. Waiting/Blocked D. New

Q1. Which scheduler selects a process from the ready queue for CPU execution?
A. Long-term scheduler B. Short-term scheduler C. Medium-term scheduler D. I/O scheduler
Answer: B

Q2. Which scheduler controls the degree of multiprogramming by admitting jobs into the system?
A. Short-term B. Long-term C. CPU scheduler D. Device scheduler
Answer: B

Q3. Which mechanism allows processes to communicate through a common memory region?
A. Message passing B. Shared memory C. Context switching D. Scheduling
Answer: B

Q4. In a parent-child relationship, which operation is commonly associated with creating a new process in Unix/Linux?
A. read() B. fork() C. close() D. write()
Answer: B

Q5. A process waiting for completion of an I/O operation is generally in which state?
A. Ready B. Running C. Waiting/Blocked D. New
Answer: C